VTC Academy
Think Ahead, Beyond Boundaries

Problem Solving & Data Organization Report

Class:	K26ISTG01
Name:	Tran Thi Huyen Trang
Student ID:	134010126013
Course:	Problem Solving & Data Organization

Table of Contents

1. INTRODUCTION **Error! Bookmark not defined.**
    1.1. Project Overview **Error! Bookmark not defined.**
    1.2. Project Objectives **Error! Bookmark not defined.**
    1.3. Project Scope **Error! Bookmark not defined.**
    1.4. Project Links **Error! Bookmark not defined.**
2. REQUIREMENTS ANALYSIS **Error! Bookmark not defined.**
    2.1. Functional Requirements **Error! Bookmark not defined.**
    2.2. Non-functional Requirements **Error! Bookmark not defined.**
3. TECHNOLOGIES USED **Error! Bookmark not defined.**
    3.1. React **Error! Bookmark not defined.**
    3.2. TypeScript **Error! Bookmark not defined.**
    3.3. Vite **Error! Bookmark not defined.**
    3.4. React Router DOM **Error! Bookmark not defined.**
    3.5. Recharts **Error! Bookmark not defined.**
    3.6. Lucide React **Error! Bookmark not defined.**
    3.7. HTML5 and CSS3 **Error! Bookmark not defined.**
    3.8. LocalStorage **Error! Bookmark not defined.**
4. SYSTEM DESIGN AND ARCHITECTURE **Error! Bookmark not defined.**
    4.1. Application Architecture **Error! Bookmark not defined.**
    4.2. Project Structure **Error! Bookmark not defined.**
    4.3. Navigation Structure **Error! Bookmark not defined.**
    4.4. Data Flow **Error! Bookmark not defined.**
5. IMPLEMENTATION AND MAIN FEATURES **Error! Bookmark not defined.**
    5.1. Dashboard **Error! Bookmark not defined.**
    5.2. Transaction Management **Error! Bookmark not defined.**
    5.3. Transaction Detail **Error! Bookmark not defined.**
    5.4. Statistics **Error! Bookmark not defined.**
    5.5. Settings **Error! Bookmark not defined.**

5.6. Responsive User Interface **Error! Bookmark not defined.**
5.7. Form Validation **Error! Bookmark not defined.**
5.8. LocalStorage Implementation **Error! Bookmark not defined.**
6. TESTING AND RESULTS **Error! Bookmark not defined.**
6.1. Testing Overview **Error! Bookmark not defined.**
6.2. Test Cases **Error! Bookmark not defined.**
6.3. Testing Results **Error! Bookmark not defined.**
7. DEPLOYMENT AND VERSION CONTROL **Error! Bookmark not defined.**
7.1. GitHub Repository **Error! Bookmark not defined.**
7.2. Vercel Deployment **Error! Bookmark not defined.**
7.3. Production Build **Error! Bookmark not defined.**
8. CONCLUSION AND LESSONS LEARNED **Error! Bookmark not defined.**
8.1. Project Conclusion **Error! Bookmark not defined.**
8.2. Lessons Learned **Error! Bookmark not defined.**
8.3. Future Improvements **Error! Bookmark not defined.**
9. APPENDIX – DEMONSTRATION VIDEO **Error! Bookmark not defined.**

1. GIỚI THIỆU ĐỀ TÀI

1.1. Lý do chọn đề tài

Trong quá trình phát triển phần mềm, thuật toán và cấu trúc dữ liệu đóng vai trò quan trọng trong việc giải quyết vấn đề và tối ưu hiệu suất chương trình.

Đề tài Problem Solving & Data Organization được thực hiện nhằm củng cố kiến thức về thuật toán, cấu trúc dữ liệu và khả năng đánh giá hiệu suất của các phương pháp giải quyết vấn đề khác nhau.

1.2. Mục tiêu

- Cài đặt và kiểm thử các thuật toán cơ bản bằng TypeScript.
- Áp dụng cấu trúc dữ liệu vào các ứng dụng thực tế.
- So sánh hiệu suất giữa hai thuật toán tìm đường BFS và A*.
- Rèn luyện kỹ năng tổ chức mã nguồn, kiểm thử và quản lý project bằng Git.

1.3. Công nghệ sử dụng

- TypeScript: Ngôn ngữ lập trình chính.
- Node.js: Môi trường chạy chương trình.
- Vitest: Công cụ kiểm thử tự động.
- Git và GitHub: Quản lý phiên bản và lưu trữ mã nguồn.

2. CẤU TRÚC PROJECT

Project được tổ chức thành các thư mục chính:

- src/algorithms: Chứa các thuật toán.
- src/data-structures: Chứa các ứng dụng cấu trúc dữ liệu.
- src/optimization: Chứa thuật toán tìm đường và tối ưu.
- tests: Chứa các file kiểm thử.
- benchmarks: Chứa chương trình đo hiệu suất.
- docs: Chứa tài liệu và hình ảnh minh chứng.

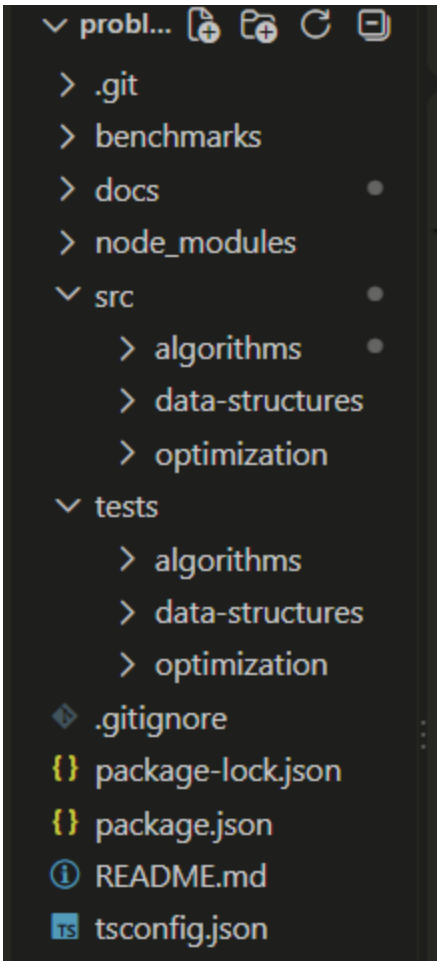


Figure1. Cấu trúc thư mục project trên VS Code

Việc phân chia mã nguồn thành từng thư mục giúp project dễ quản lý, kiểm thử và mở rộng.

3. PHẦN 1 – CÀI ĐẶT THUẬT TOÁN

Project triển khai năm thuật toán cơ bản.

3.1. Prime Number

Thuật toán tìm số nguyên tố thứ N. Chương trình kiểm tra tính nguyên tố của các số và xác định số nguyên tố ở vị trí được yêu cầu.

3.2. Reverse String

Thuật toán đảo ngược chuỗi mà không sử dụng phương thức reverse() có sẵn. Việc triển khai giúp hiểu cách xử lý chuỗi và truy cập các ký tự.

3.3. Palindrome

Thuật toán kiểm tra chuỗi đối xứng. Chương trình xử lý khoảng trắng và dấu câu trước khi so sánh các ký tự từ hai đầu chuỗi.

3.4. Kth Largest

Thuật toán tìm phần tử lớn thứ K không trùng lặp trong mảng. Giải pháp sử dụng Min Heap để quản lý các phần tử cần thiết.

3.5. Character Frequency

Thuật toán đếm số lần xuất hiện của từng ký tự trong chuỗi bằng cấu trúc Map.

4. PHẦN 2 – ỨNG DỤNG CẤU TRÚC DỮ LIỆU

Project triển khai ba ứng dụng thực tế.

4.1. Contact Manager

Ứng dụng quản lý danh bạ sử dụng Hash Map (Map).

Các chức năng:

- Thêm liên hệ.
- Tìm kiếm liên hệ.
- Cập nhật thông tin.
- Xóa liên hệ.

Việc sử dụng Map giúp truy xuất dữ liệu theo khóa thuận tiện và hiệu quả.

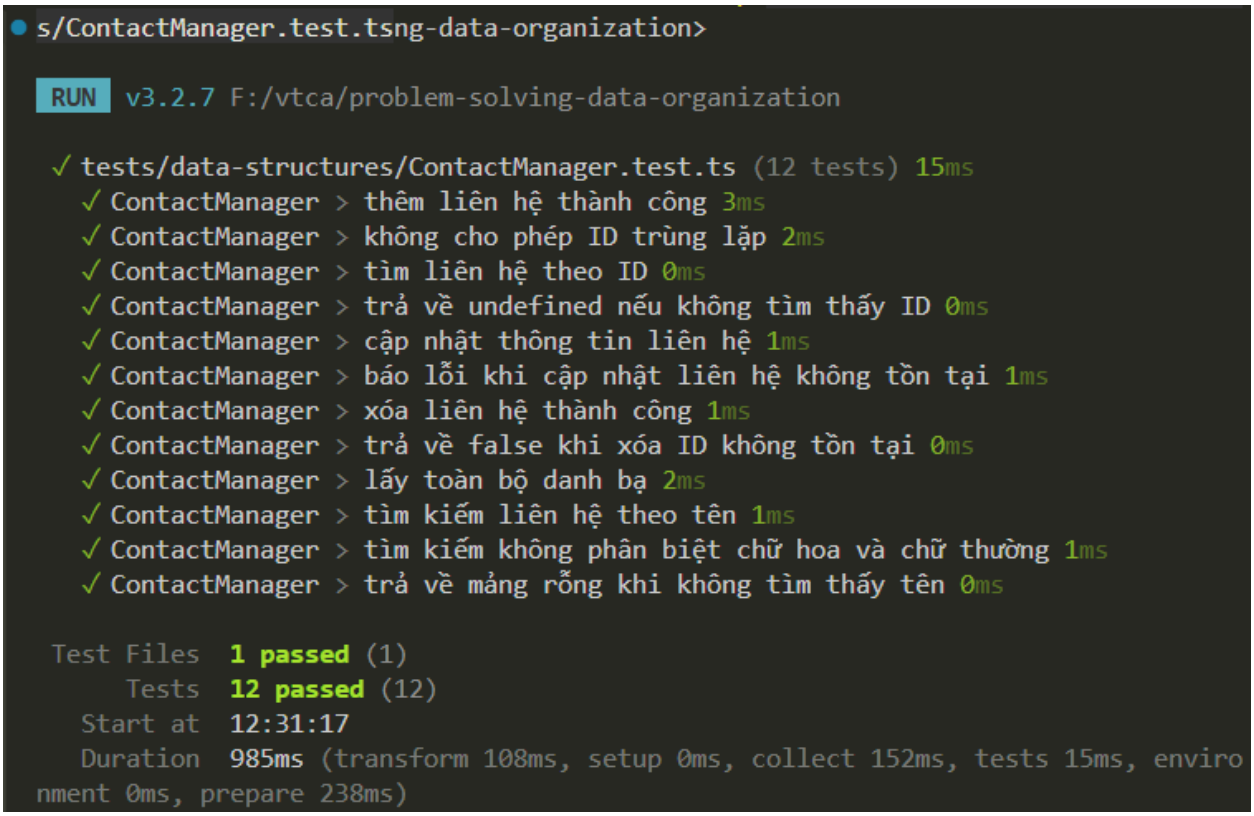


Figure 2. Kiểm thử Contact Manager

4.2. Game Leaderboard

Ứng dụng bảng xếp hạng trò chơi sử dụng Max Heap kết hợp Hash Map.

Các chức năng:

- Thêm người chơi.
- Cập nhật điểm số.
- Tìm kiếm người chơi.
- Hiển thị danh sách người chơi có điểm cao nhất.

Max Heap hỗ trợ lấy người chơi có điểm cao, trong khi Hash Map hỗ trợ quản lý người chơi theo định danh.

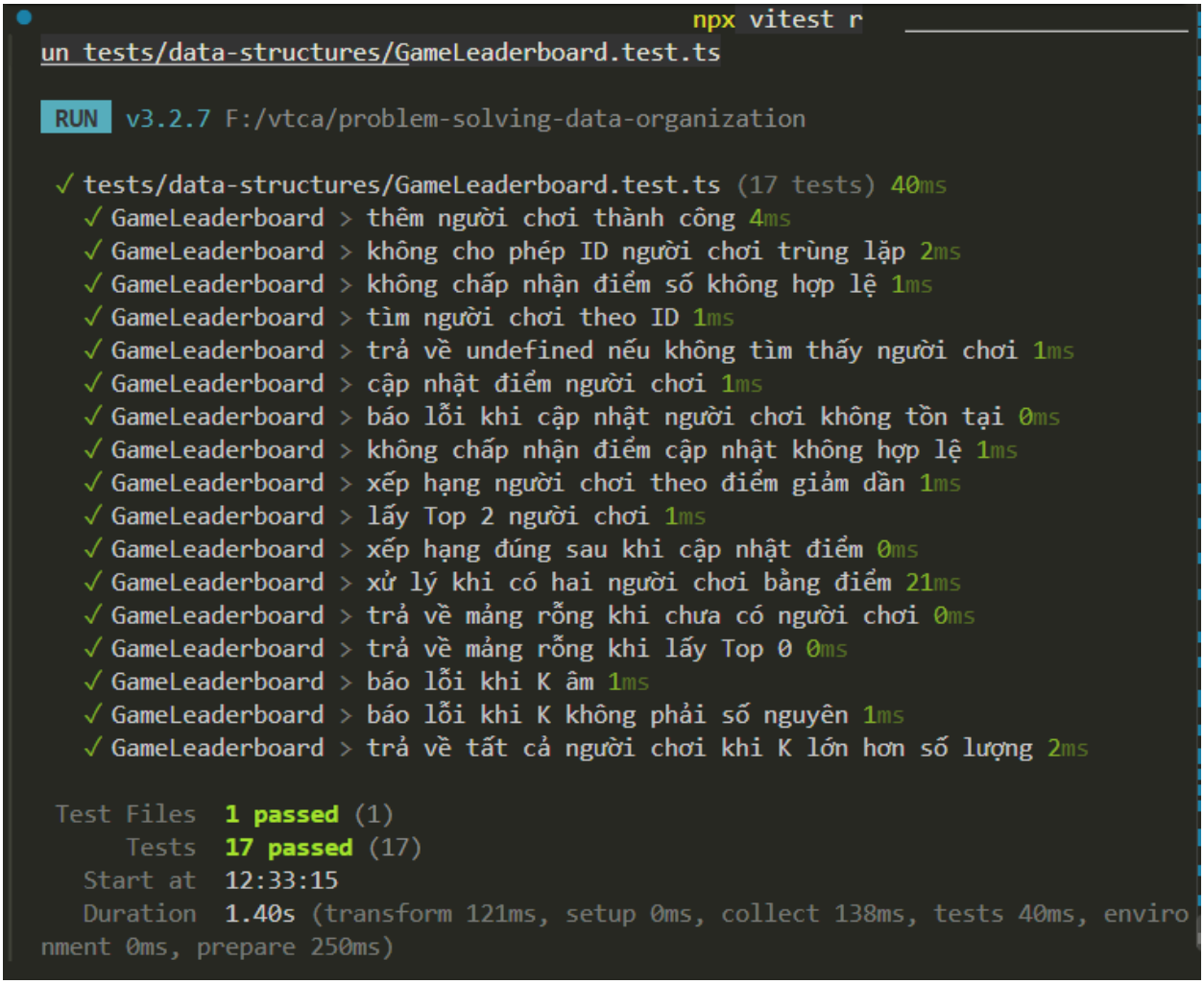


Figure 3. Kiểm thử Game Leaderboard

4.3. LRU Cache

Ứng dụng bộ nhớ đệm sử dụng Hash Map và Doubly Linked List.

Các chức năng:

- Thêm dữ liệu vào cache.
- Truy xuất dữ liệu.
- Cập nhật thứ tự sử dụng gần nhất.
- Loại bỏ phần tử ít được sử dụng gần đây nhất khi cache đầy.

Sự kết hợp giữa hai cấu trúc dữ liệu giúp các thao tác truy xuất, cập nhật và loại bỏ phần tử đạt độ phức tạp O(1) trung bình.

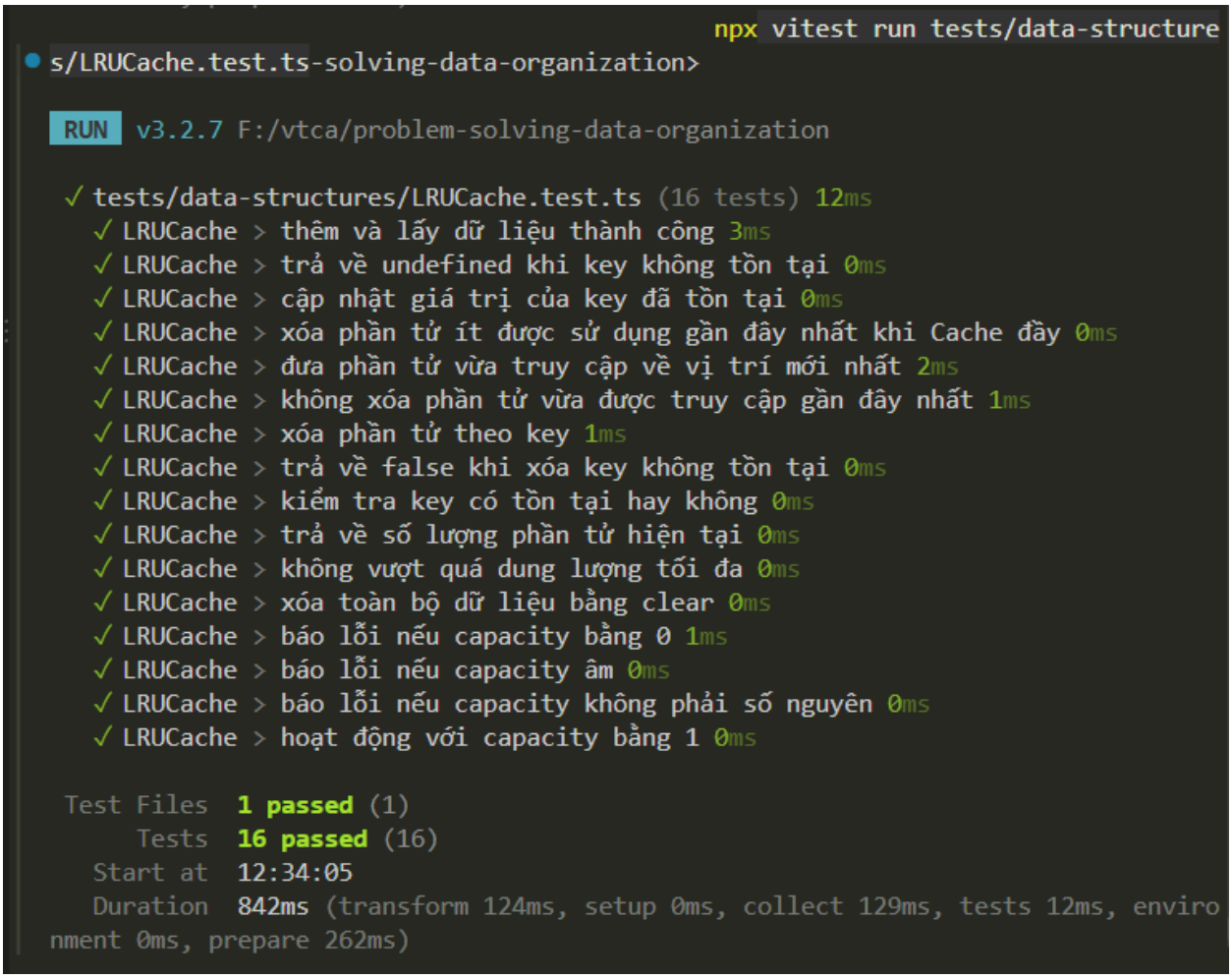


Figure 4. Kiểm thử LRU Cache

5. PHẦN 3 – TỐI ƯU THUẬT TOÁN BFS VÀ A*

5.1. Bài toán

Bài toán yêu cầu tìm đường đi ngắn nhất từ điểm bắt đầu đến điểm đích trong mê cung dạng ma trận hai chiều.

Quy ước:

- 0: Ô có thể di chuyển.
- 1: Ô có vật cản.
- Chỉ được di chuyển theo bốn hướng: lên, xuống, trái và phải.

5.2. Thuật toán BFS

BFS (Breadth-First Search) tìm kiếm theo chiều rộng, lần lượt duyệt các ô lân cận theo từng lớp khoảng cách.

Với đồ thị có V đỉnh và E cạnh, độ phức tạp thời gian là O(V + E).

Ưu điểm:

- Đảm bảo tìm được đường đi ngắn nhất trong đồ thị không trọng số.
- Cách triển khai tương đối đơn giản.

Nhược điểm:

- Có thể phải duyệt nhiều ô không cần thiết.

5.3. Thuật toán A*

A* sử dụng hàm đánh giá:

f(n) = g(n) + h(n)

Trong đó:

- g(n): Chi phí từ điểm bắt đầu đến nút hiện tại.
- h(n): Ước lượng chi phí từ nút hiện tại đến đích.

Project sử dụng khoảng cách Manhattan làm heuristic vì chỉ cho phép di chuyển theo bốn hướng.

Ưu điểm:

- Có thể giảm số lượng nút cần duyệt nhờ heuristic.
- Hữu ích khi cần tìm đường đến một đích cụ thể.

Nhược điểm:

- Cần thêm chi phí tính heuristic và quản lý hàng đợi ưu tiên.
- Thời gian chạy thực tế không phải lúc nào cũng nhanh hơn BFS.

5.4. Kết quả benchmark

Thí nghiệm được thực hiện trên mê cung kích thước 60 × 60 với 15 lần chạy.

Chỉ số	BFS	A*
Thời gian trung bình	4.5005 ms	6.7445 ms
Thời gian trung vị	4.6417 ms	6.6559 ms
Số nút đã duyệt	2730	2475
Độ dài đường đi	591	591

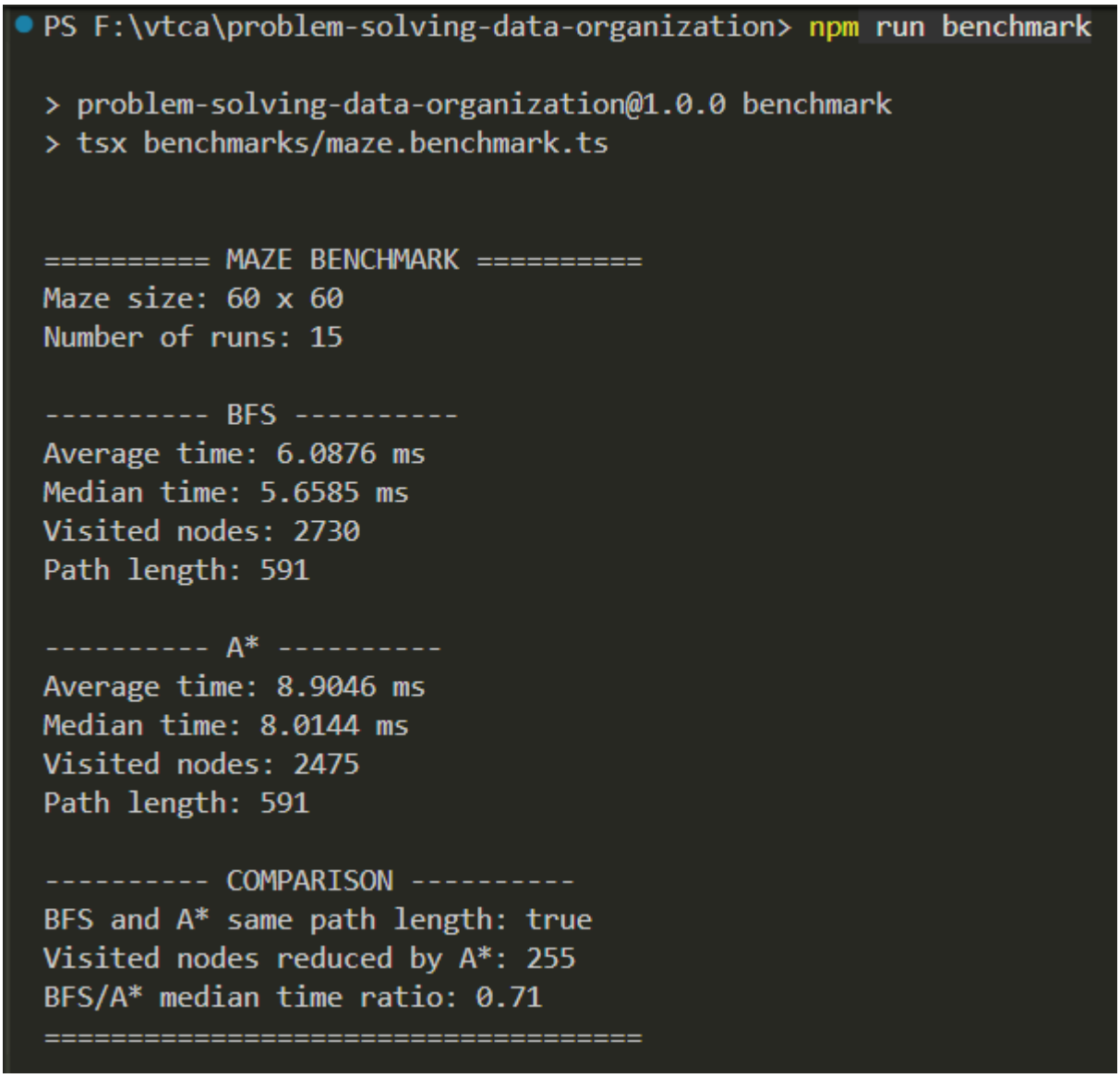


Figure 5. Kết quả benchmark BFS và A*

5.5. Nhận xét kết quả

Hai thuật toán đều tìm được đường đi có độ dài 591 ô.

A* duyệt ít hơn BFS 255 nút, tương đương giảm khoảng 9,34% số nút đã duyệt.

Tuy nhiên, thời gian trung vị của BFS là 4.6417 ms, thấp hơn thời gian 6.6559 ms của A*. Nguyên nhân có thể do A* cần thực hiện thêm các phép tính heuristic và thao tác trên hàng đợi ưu tiên.

Kết quả cho thấy số lượng nút được duyệt ít hơn không đồng nghĩa với thời gian thực thi luôn nhanh hơn. Hiệu suất còn phụ thuộc vào cấu trúc mê cung và chi phí xử lý của từng thuật toán.

6. KIỂM THỬ VÀ ĐÁNH GIÁ

Project sử dụng Vitest để thực hiện kiểm thử tự động.

Các nhóm kiểm thử bao gồm:

- Kiểm tra kết quả của năm thuật toán.
- Kiểm tra các chức năng của Contact Manager.
- Kiểm tra các thao tác của Game Leaderboard.
- Kiểm tra hoạt động của LRU Cache.
- Kiểm tra khả năng tìm đường và xử lý trường hợp không có đường đi của BFS và A*.

Ngoài ra, project sử dụng TypeScript Compiler để kiểm tra kiểu dữ liệu và chạy chương trình benchmark nhằm đánh giá hiệu suất.

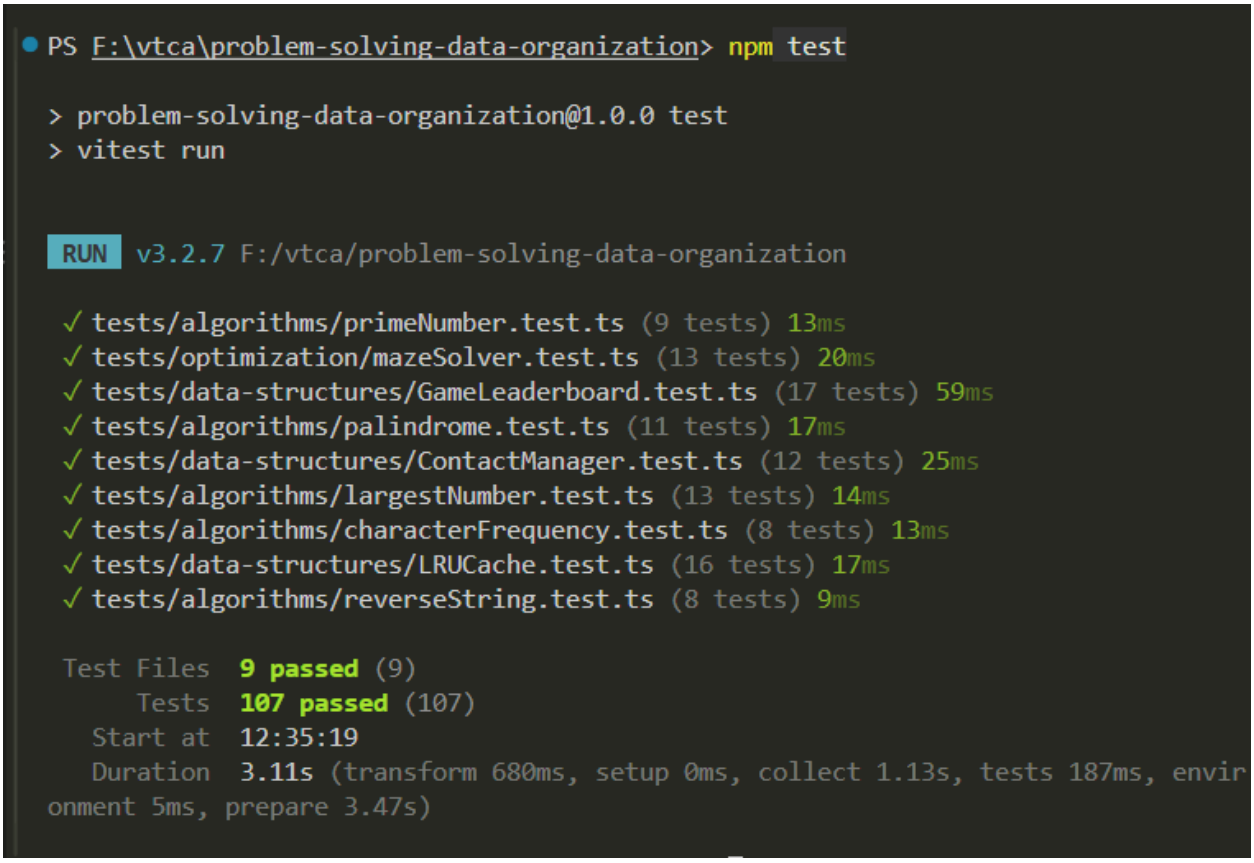


Figure 6. Kết quả npm test

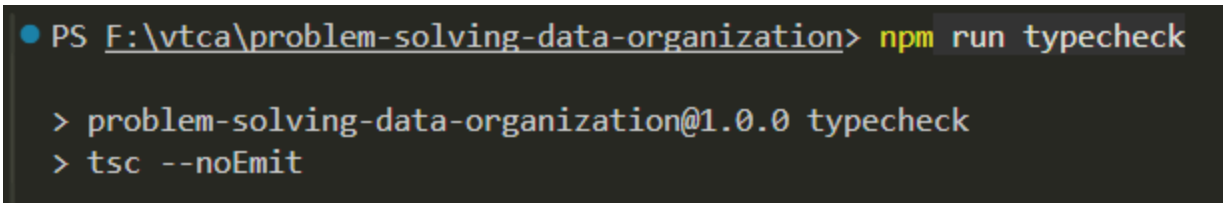


Figure 7. Kết quả npm run typecheck

7. KHÓ KHĂN VÀ BÀI HỌC KINH NGHIỆM

Trong quá trình thực hiện, một số khó khăn gặp phải bao gồm:

- Lựa chọn cấu trúc dữ liệu phù hợp cho từng ứng dụng.
- Xử lý các trường hợp đầu vào đặc biệt và dữ liệu không hợp lệ.
- Đảm bảo thuật toán tìm đường trả về kết quả chính xác.
- Viết các trường hợp kiểm thử để phát hiện lỗi.
- Đánh giá hiệu suất dựa trên nhiều chỉ số khác nhau.

Thông qua project, tôi hiểu rõ hơn cách áp dụng thuật toán và cấu trúc dữ liệu vào các tình huống thực tế. Đồng thời, tôi cải thiện kỹ năng viết TypeScript, tổ chức mã nguồn, kiểm thử tự động và sử dụng Git để quản lý phiên bản.

8. KẾT LUẬN

Project đã triển khai các thuật toán cơ bản, ứng dụng cấu trúc dữ liệu và so sánh hai thuật toán tìm đường BFS và A*.

Thông qua quá trình thực hiện, tôi đã củng cố kiến thức về thuật toán, cấu trúc dữ liệu và kiểm thử phần mềm. Kết quả benchmark cũng cho thấy cần đánh giá thuật toán dựa trên cả thời gian thực thi và số lượng nút được duyệt thay vì chỉ dựa vào một chỉ số.

GitHub Repository: https://github.com/huyentrang277/problem-solving-data-organization.git